GenAI Used: Gemini (accessed on KU Linux machine using Firefox)

Prompt Given:

{Assignment5.cpp included as attachment}

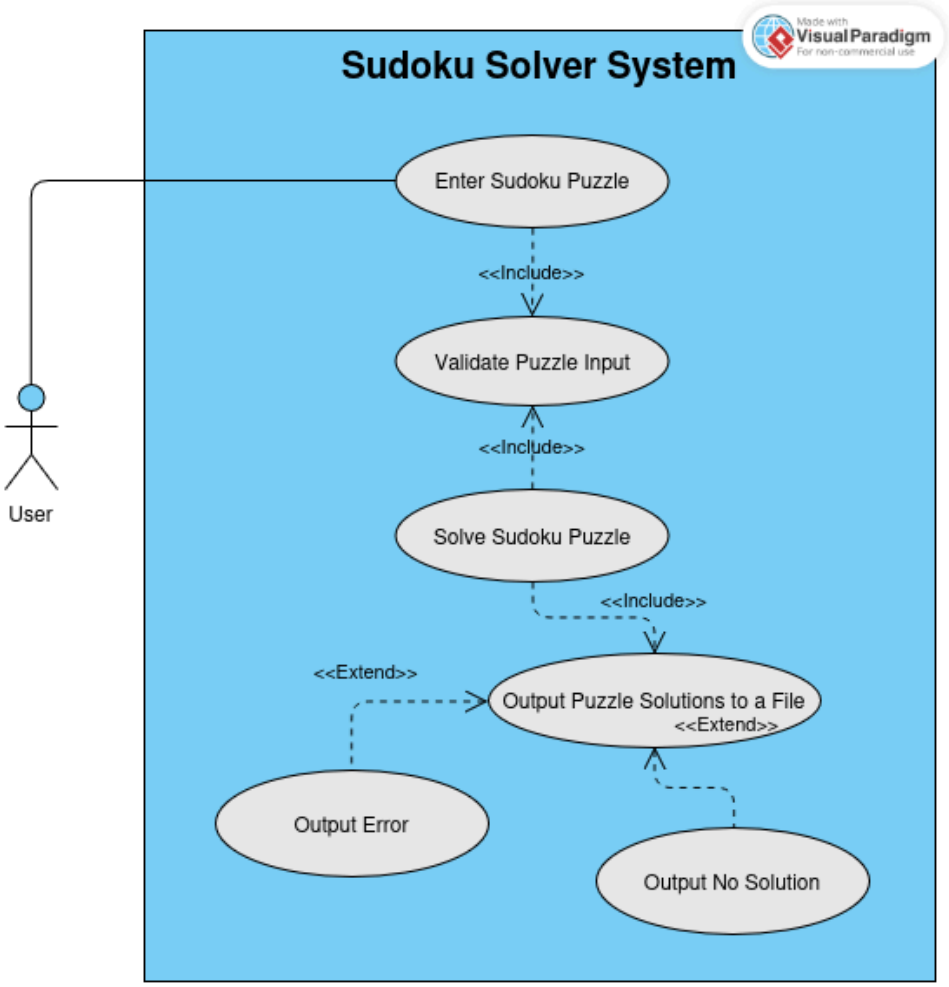
Attached is a C++ program that solves sudoku puzzles from text files. When the program solves a puzzle, it writes the result to another text file.

Given the program, create a UML Use Case Diagram using these steps for each use case involved in the overall program:

a. Identify/discover a use case
b. Provide a brief description for the use case
c. Provide a step-by-step flow (outline) of the events for the use case
d. Provide a detailed description for each step of the use case outline

Use cases that already exist include: Enter Sudoku Puzzle, Validate Puzzle Input, and Solve Sudoku Puzzle. Define additional use cases that describe the process of outputting the puzzle solutions to a file using the description steps mentioned above.

Graphic made with Visual Paradigm using GenAI output and previous UML Use Case diagram as guide:



➜ Use Case: Enter Sudoku Puzzle
- Brief Description:
  - The program takes puzzles from text files named (puzzle1.txt - puzzle5.txt).
- Outline:
  - User runs program
  - The program obtains files
- Detailed Description:
  - The user runs the program from the terminal with the executable.
  - The program obtains the files (puzzle1.txt - puzzle5.txt) by defining the names.

➜ Use Case: Validate Puzzle Input
- Brief Description:
  - The program reads each cell in the puzzle file and checks to make sure the number of cells, dimensions, and numbers are all valid to prepare for solving.
- Outline:
  - Open file
  - Check cell
  - Determine type
  - Check dimensions
- Detailed Description:
  - The program opens the file in the order specified in the Enter Sudoku Puzzle use case. If the file cannot be opened, the program outputs an error.
  - The program checks the cell to make sure it is either an underscore or a number.
  - The program uses the amount of cells checked to see if the amount is right (checking dimensions).

➜ Use Case: Solve Sudoku Puzzle
- Brief Description:
  - The program solves the sudoku puzzle by using a DFS algorithm with backtracking.
- Outline:
  - Verify sudoku grid
  - Pick best empty cell
  - Find best candidate number
- Detailed Description:
  - Verify that the sudoku grid is valid/consistent for solving
  - Pick the best empty cell to begin the solving process with
  - Recursively try each candidate number, and backtrack when necessary

➜ Use Case: Output Solutions to a File

- Brief Description:
  - The program outputs the result of the puzzle solving process to a text file.
- Outline:
  - Create solution file
  - Write puzzle solution to file
- Detailed Description:
  - The program creates the solution file and names it accordingly by taking the digit from the puzzle file.
  - If the sudoku puzzle file cannot be opened or its configuration is not valid, write the error to the file.
  - If the program finds a solution or solutions to the puzzle, write each solution and the total number of solutions to the puzzle.
  - If there is no solution, write that there is no solution to the file.